Graph Neural Network Adapters for Multi-Label Chest X-Ray Classification Without VLM Fine-Tuning: Frozen Scorers, Structured Heads, and Leakage-Free Calibration

Repository: MBZAI multi-label CXR pipeline **Codebase:** `scripts/01–13`, `configs/`, `gradio_inference.py` **Dataset:** CheXpert-v1.0-small (Frontal + Lateral) **Label space:** {`Atelectasis`, `Cardiomegaly`, `Effusion`, `Pneumonia`, `Edema`, `Consolidation`, `No Finding`} ($C = 7$)

Positioning. The central proposal is **domain adaptation for multi-label classification without fine-tuning the VLM**: keep the foundation model frozen (one-time inference or API scores), and learn a **small graph-based adapter** that maps VLM outputs—and optionally a second frozen image encoder—into calibrated logits for the target domain (here, CheXpert-style labels and masking). That trades **full VLM fine-tuning** (GPU memory, catastrophic forgetting, data-hungry updates) for **adapter training** on cheap tabular tensors $(x_{\text{logits}}, x_{\text{probs}}, y_{\text{true}}, y_{\text{mask}})$ plus optional CLIP embeddings.

Abstract

We **propose graph neural network (GNN) adapters as a practical alternative to end-to-end VLM fine-tuning** for multi-label chest X-ray classification: the VLM stays frozen while a lightweight head performs **domain-specific calibration and structured reasoning** over labels. Concretely, we re-calibrate a frozen Qwen2-VL-2B-Instruct baseline on CheXpert and compare four adapters of increasing structure: (i) an MLP over VLM logit/probability vectors; (ii) a residual label-graph GNN on a *co-error* adjacency mined from training disagreements; (iii) a homogeneous CLIP-conditioned label-graph GNN; and (iv) a bipartite *attribute → object* GNN with frozen CLIP object features and VLM attribute nodes. All adapters use masked, class-weighted binary cross-entropy on patient-grouped splits and are evaluated under (a) fixed $t = 0.5$ and (b) a leakage-free 4-way *train/calib/val/test* protocol with per-class thresholds tuned **only** on `calib`. The bipartite CLIP variant (`gnn13_clip_bipartite`) reaches the best calibrated test macro-F1 of **0.6889**, **+3.4** over the calibrated MLP (**0.6544**) and **+3.8** over the same leakage-free calibrated reference applied to frozen VLM probabilities (**0.6512**; i.e. exported `x_probs` with thresholds fit on `calib` only—see §6.1). Naive $t = 0.5$ macro-F1 on the frozen model remains ≈ 0.047 because logits are systematically negative off `No Finding`. We also show how threshold tuning on the same split as evaluation can inflate macro-F1 for poorly calibrated logits—a methodological artifact, not model capacity—and document how to avoid it with a held-out `calib` split (§6.2).

---

1. Introduction

Thesis. Fine-tuning a large VLM for every new hospital, label set, or policy change is often **prohibitively expensive**: it requires gradient updates through billions of

parameters, careful replay or regularization to limit forgetting, and substantial curated data. A complementary strategy is **post-hoc domain adaptation at the decision layer**: treat the VLM as a fixed feature generator $f_{\text{VLM}}(x) \mapsto (z, p) \in \mathbb{R}^{2C}$ (and optionally a second frozen encoder $e_{\text{CLIP}}(I)$), then learn a small module g_θ with $|\theta| \ll |\text{VLM}|$ so that $\hat{z} = g_\theta(z, p, e)$ matches the target domain's labels and masking. **GNN-based** g_θ is attractive because multi-label CXR findings are **not independent**: co-occurrence, mutual exclusion, and systematic error patterns are naturally expressed as **message passing on a label graph** or as **attribute→image bipartite** structure—inductive biases that a flat MLP must rediscover from data alone.

Multi-label classification of chest X-rays under the CheXpert label policy still presents three coupled difficulties even under this adapter framing: (1) extreme **class imbalance**, (2) **uncertain (-1) labels** that must be policy-mapped, and (3) **miscalibrated** zero-shot VLM probabilities (often dominated by `No Finding` at naive thresholds). We keep the VLM frozen and ask three empirical questions that stress-test the **GNN-as-domain-adapter** idea:

1. **Is structure useful?** Does encoding label co-occurrence/co-error as a graph help over a flat MLP that already sees the full $[\, x_{\text{logit}};\, x_{\text{prob}} \,]$ vector?
2. **Is the image useful again?** Once we already have VLM scores, is there marginal value in re-injecting a *different* frozen image encoder (CLIP)?
3. **Is the wiring useful?** Does the bipartite *attribute → object* topology provide a better inductive bias than a homogeneous label graph?

To answer these in a way that is **not contaminated by threshold-tuning leakage**, we build a 4-way patient-grouped split (`train_fit / calib / val / test`) and frozen per-class thresholds that are *only* tuned on `calib` and then re-applied unchanged to `val` and `test`. All five models in our comparison (zero-shot, MLP, three GNNs) are evaluated under this identical protocol.

1.1 What "domain adaptation without finetuning" buys you

Aspect	Full VLM fine-tuning	GNN (or MLP) adapter on frozen (z, p)
Trainable parameters	10^9–10^{10}	10^4–10^6 in this repo
GPU memory at train	Very high (vision + LM)	Low (small tensors + optional CLIP cache)
Forgetting / drift	Risk when updating foundation	None on the VLM; only θ changes
New label policy or site	Often re-finetune	Re-align JSONL + retrain adapter + retune thresholds
Domain knowledge	Implicit in gradients	Explicit in graph topology (co-error, bipartite)

The **GNN variants** add one more lever: **structured transfer**—sharing statistical strength across labels via edges—without ever backpropagating into the VLM. That

is the sense in which this work is a **proposal for cheap domain adaptation** rather than a claim that GNNs always beat MLPs (under fair calibration, residual label-graph and MLP are close; CLIP + bipartite is where structure pays).

Contributions

- **Conceptual proposal: GNN-based adapters as a frozen-VLM alternative to fine-tuning** for multi-label CXR: adapt the *output distribution and label dependencies* to the target domain while the foundation model remains a fixed scorer.
- **Adapter family** spanning four distinct inductive biases over the same frozen VLM outputs (`vlm_zeroshot`, `vlm_mlp`, `gnn07_label_residual`, `gnn12_clip_vlm_homo`, `gnn13_clip_bipartite`), each registered in a small model registry (`scripts/model_registry.p` for organized, reproducible run management.
- **Co-error label graph**: a sparsified, row-normalized directed adjacency built from VLM disagreements on the training split (`scripts/04_build_coerror_graph.py`), which serves as the structural prior for the residual and homogeneous GNNs.
- **Bipartite NativeGNN**: a CLIP-object / VLM-attribute message-passing network with optional VLM-positive edge masking (`scripts/gnn_bipartite.py`, `scripts/13_train_bipartite_gnn_adapter.py`).
- **Fairness-aware evaluation**: a 4-way calibrated protocol (`scripts/03_make_multilabel_spli` `scripts/08_tune_thresholds.py`, `scripts/09_evaluate_test.py`) that quantifies and removes the threshold-tuning leakage that otherwise inflates GNN scores by tens of F1 points.
- **Empirical findings** reported in §6, with a root-cause analysis (§7) of the most common evaluation pitfalls and the genuine gains of CLIP injection and bipartite wiring.

---

2. Related Work

CheXpert and uncertain labels. CheXpert (Irvin et al., 2019) introduced the U-Ones / U-Zeros / U-Ignore policies for the `-1` (uncertain) label. We use **U-Zeros** with explicit masking: empty entries set both `y` and `mask` to 0, so excluded samples contribute nothing to the loss or to per-class F1.

Frozen-foundation adapters. A growing body of work freezes a large pretrained encoder and trains a small adapter for downstream tasks (LoRA, prefix tuning, linear probes). For multi-label classification specifically, Chen et al. (2019, ML-GCN) proposed using a label-correlation graph to refine class-wise predictions. Our `gnn07` variant is a streamlined residual realization of this idea: nodes are labels, features are per-row $(\text{logit}_i, \text{prob}_i)$, and a single message-passing step adds a residual on top of the VLM logits.

CLIP for medical imaging. CLIP (Radford et al., 2021) is not a chest-X-ray-specialised encoder, but its image embeddings are a cheap, complementary signal to those of any other VLM. We use a frozen `openai/clip-vit-base-patch32` image

branch and project it into the GNN node space (`gnn12`) or into a dedicated *object* node (`gnn13`).

Bipartite attribute graphs. Encoding *instances* and *targets* as different node roles is classic in heterogeneous and relational graph networks: message passing can traverse **bipartite incidence structure** rather than enforcing a homogeneous $C \times C$ label graph everywhere. Foundations include **relational GCN** for typed edges between entity kinds (Schlichtkrull et al., 2018), **semantic-level meta-path attentive aggregation** across multiple node roles (Wang et al., 2019, HAN), and the modular **Neural Message Passing** view in which pairwise updates generalize heterogeneous interaction patterns (Gilmer et al., 2017). Our bipartite instantiation places **one CLIP-derived object node per image** and C **VLM-attribute nodes**, with weighted-mean attribute→object pooling before readout—a lightweight inductive bias aligned with CXR semantics without depending on PyG (`scripts/gnn_bipartite.py`).

Threshold calibration. Per-label operating points tie directly to probabilistic calibration and **proper scoring** viewpoints (Niculescu-Mizil & Caruana, 2005; Guo et al., 2017). Selecting thresholds by grid search remains common in multi-label setups (Lipton et al., 2014 optimise F-measure via threshold choices). Critically: **fitting thresholds—or any discrete decision rule—on the same split later used as a "selection" metric** induces optimistic bias (Varma & Simon, 2006; Cawley & Talbot, 2010). Our **`calib`** split mirrors the textbook fix used in unbiased model comparison: isolate post-hoc threshold fitting on disjoint data, unchanged when applied to validation and held-out test. §6 quantifies how badly that discipline matters when logits are saturated.

---

3. Dataset and Preprocessing

3.1 Source

We use **CheXpert-v1.0-small** with the standard `train.csv` (≈223k rows after header) and `valid.csv` (234 rows). Both are CSVs with one row per study + view; each row contains 14 finding columns. We restrict to the 7-label canonical space below.

3.2 Canonical label mapping (`scripts/common_multilabel.py`)

```
"Atelectasis"     -> "Atelectasis"
"Cardiomegaly"    -> "Cardiomegaly"
"Pleural Effusion"-> "Effusion"
"Pneumonia"       -> "Pneumonia"
"Edema"           -> "Edema"
"Consolidation"   -> "Consolidation"
"No Finding"      -> "No Finding"
```

Uncertain (-1) labels are mapped via `parse_uncertain(value, policy="u_zeros")`: `1` → `(1,1)`, `0` → `(0,1)`, `-1` → `(0,1)` under U-Zeros, and empty → `(0,0)`. The `mask=0` entries are excluded from both loss and F1 computation, which is the single most important detail for honest multi-label evaluation on CheXpert.

3.3 VLM alignment (`scripts/02_align_vlm_outputs.py`)

We score every image once with **Qwen2-VL-2B-Instruct** (zero-shot, structured-prompt-and-parse) and persist `{path, scores: {label: prob}}` to JSONL shards under `data/outputs_vlm_corrected/`. Alignment joins on `normalize_path(path)` and produces aligned rows of the form

```
{path, image_id, patient_id,
 x_probs   in R^7,    % in [0,1]
 x_logits  in R^7,    % safe_logit(x_probs), eps 1e-6
 y_true    in {0,1}^7,
 y_mask    in {0,1}^7}
```

This `(x_logits, x_probs, y_true, y_mask)` quadruple is the only thing the four downstream adapters see during training; the VLM is never re-invoked at adapter-train time.

3.4 Splits

Two split protocols co-exist in the repo, sharing the same row schema and the same `patient_id` grouping (no patient overlap across splits).

(a) 3-way `default` (`scripts/03_make_multilabel_splits.py`): 70 / 15 / 15 patient-level shuffle, seed 42. Used for all `protocol=default` numbers.

(b) 4-way `calibrated4way` (`scripts/03_make_multilabel_splits_4way.py`): 70 / 10 / 10 / 10 patient-level shuffle, seed 42 → `train_fit / calib / val / test`. The `calib` split exists *only* to host per-class threshold tuning; `val` and `test` are entirely unseen by the threshold optimizer.

Multi-label intermediates produced by these scripts carry on the order of **44k train** rows under the patient-grouped regimes used here (exact counts drift slightly with `02_align_vlm_outputs.py` parity); registry runs report **43 778 train / 9 357 val / 9 197 test** for `protocol=default` and analogous 4-way sizes for `calibrated4way` (see `metrics.json` `dataset_sizes` under each run).

3.5 Co-error label graph (`scripts/04_build_coerror_graph.py`)

For every training row with at least one positive label, we form

$$\text{present}(r) = \{i : y_i = 1 \wedge m_i = 1\}, \qquad \text{absent}(r) = \{j : y_j = 0 \wedge m_j = 1\},$$

and accumulate

```
M[i, j] += 1   for i in present sorted ASC by VLM prob (top_k=3)
                for j in absent  sorted DESC by VLM prob (top_k=3), j != i
```

i.e. an edge `i → j` is added whenever the VLM **missed** a true positive `i` while **hallucinating** a likely false positive `j`. Rows of `M` are L1-normalized to give `W`, then sparsified per-source to the top-`k=3` targets with weight `≥ τ=0.02`. The resulting `(edge_index, edge_weight)` defines an asymmetric directed graph stored in

`data/processed/graph/{edge_index,edge_weight}.json`. The same graph is reused, identically, by `gnn07` and `gnn12`.

---

4. Methodology

4.1 Problem and notation

For a single image with VLM outputs $z = \text{logits} \in \mathbb{R}^C$, $p = \sigma(z) \in [0,1]^C$, label vector $y \in \{0,1\}^C$ and mask $m \in \{0,1\}^C$, each adapter outputs **calibrated logits** $\hat{z} \in \mathbb{R}^C$ and is trained with masked, class-weighted BCE:

$$\mathcal{L}(\hat{z}, y, m) = \frac{\sum_{i=1}^{C} m_i \cdot \text{BCEWithLogits}(\hat{z}_i, y_i;\, w_i^+)}{\sum_{i=1}^{C} m_i + \varepsilon}, \qquad w_i^+ = \min\Big(\tfrac{N_i^-}{\max(N_i^+,1)},\, 100\Big).$$

N_i^+, N_i^- are training positive/negative counts on the (masked) train split; w^+ is the `pos_weight` used by `binary_cross_entropy_with_logits`.

4.2 M0 — `vlm_zeroshot` (`VLMZeroShot`)

$$\hat{z} = z, \quad \hat{p} = \sigma(z)$$

No learnable parameters. Decision rule: $\hat{y}_i = \mathbf{1}[\hat{p}_i \geq 0.5]$. Implemented in `scripts/05_run_baseline_frozen_vlm.py`.

4.3 M1 — `vlm_mlp` (`VLMFeatureMLP`)

A flat 2-layer MLP over the concatenated logit/prob vector:

$$x = [\, z_1, p_1, z_2, p_2, \ldots, z_C, p_C \,] \in \mathbb{R}^{2C}, \qquad \hat{z} = W_2\, \text{Dropout}(\text{ReLU}(W_1 x)),$$

with $W_1 \in \mathbb{R}^{64 \times 2C}$, $W_2 \in \mathbb{R}^{C \times 64}$, AdamW, `lr=1e-3`, `weight_decay=1e-4`, 20 epochs, dropout 0.1. Implemented in `scripts/06_run_baseline_mlp.py`. This is the simplest baseline that can learn class-specific *bias correction* and per-label *temperature*.

4.4 M2 — `gnn07_label_residual` (`LabelGraphResidualGNN`)

C-node homogeneous graph with row-normalized adjacency $A \in \mathbb{R}^{C \times C}$ (built in `build_adj` by adding self-loops then row-stochastic normalization). Per-row node features are 2-d: $h_i^{(0)} = [z_i,\, p_i]$. The model is a tiny per-node MLP followed by a single message-pass and a residual on the original logits:

$$g_i = W_2\, \text{ReLU}(W_1 h_i^{(0)}) \in \mathbb{R}, \quad \Delta = g\, A^\top \in \mathbb{R}^C, \quad \hat{z} = z + \alpha\, \Delta.$$

`hidden_dim=32`, `alpha=0.5`, `lr=3e-4`, AdamW with cosine LR + 2-epoch warmup, gradient clip 1.0, 80 epochs max with `early_stop_patience=18` on `val_bce`. Implemented in `scripts/07_train_gnn_adapter.py`.

4.5 M3 — `gnn12_clip_vlm_homo` (`ClipVlmHomogeneousGNN`)

Same homogeneous label graph as M2, but each node now also sees a projected CLIP image embedding. With $e \in \mathbb{R}^{D_{\text{clip}}}$ the frozen CLIP image embedding (`openai/clip-vit-base-patch32`):

$$\tilde{z} = \text{ReLU}(W_e e) \in \mathbb{R}^H, \qquad h_i^{(0)} = \text{ReLU}\big(W_n[\tilde{z};\, z_i;\, p_i]\big).$$

Then K GNN layers with normalized adjacency A (elements A_{ij}):

$$h_i^{(k+1)} = \text{ReLU}\Big(W^{(k)} \sum_j A_{ij}\, h_j^{(k)}\Big), \qquad \Delta_i = W_h h_i^{(K)} \in \mathbb{R}, \qquad \hat{z}_i = z_i + \alpha\, \Delta_i \quad (i = 1, \dots, C).$$

Writing $\Delta = (\Delta_1, \dots, \Delta_C)^\top$ yields $\hat{z} = z + \alpha\, \Delta \in \mathbb{R}^C$ componentwise.

`hidden_dim=64`, `gnn_layers=2`, `alpha=0.5`, `lr=3e-4`, AdamW + cosine, batch 32, 60 epochs, early-stop 16. Implemented in `scripts/12_train_clip_vlm_gnn_adapter.py`. CLIP embeddings are precomputed once per split and cached as a single `.pt` (`--clip_cache_pt`).

4.6 M4 — `gnn13_clip_bipartite` (`ClipBipartiteAttributeGNN`)

A **bipartite** graph with C attribute nodes (one per label) and one object node (the image). Attribute features are still $[z_i, p_i]$; the object feature is a linear projection of CLIP: $o^{(0)} = W_{\text{clip}} e$. Each bipartite layer aggregates a weighted-mean message from attributes to the object, projects, concatenates with the object state, then updates with a small MLP + dropout (`scripts/gnn_bipartite.py::BipartiteMessagePassingLayer`):

$$\mu = \frac{\sum_i w_i\, W_{\text{am}}\, a_i}{\sum_i w_i + \varepsilon}, \quad \nu = W_{\text{ap}}\, \mu, \quad o^{(k+1)} = \text{Dropout}\big(\text{ReLU}\big(W_u[o^{(k)};\, \nu]\big)\big).$$

After L layers (`hidden_dims=[512,256]`), a single classifier head produces C logits from the object state, and a residual term re-anchors them to the VLM:

$$\hat{z} = W_{\text{cls}}\, o^{(L)} + \alpha\, z, \qquad \alpha = 0.5.$$

The edge weights w_i come from `build_bipartite_edge_weights(p, mode, τ)`:

- `mode=all`: $w_i = 1$ (uniform mean).
- `mode=vlm_positive`: $w_i = \mathbf{1}[p_i \geq \tau]$, with all-ones fallback if a row has no edges.

`object_feature_dim=512`, dropout 0.2, `lr=3e-4`, AdamW + cosine, batch 32, 60 epochs, early-stop 16. Implemented in `scripts/13_train_bipartite_gnn_adapter.py`.

4.7 Why these four shapes

Adapter	Sees image again?	Uses label graph?	Uses CLIP?	Topology
`vlm_mlp`	no	no	no	dense flat
`gnn07_label_residual`	no	yes ($A_{C\times C}$)	no	homogeneous
`gnn12_clip_vlm_hybrid`	yes	yes ($A_{C\times C}$)	yes	homogeneous, image-broadcast
`gnn13_clip_bipartite`	yes	implicit via attr→obj	yes	bipartite

Each row adds **exactly one** capability over the previous one, which lets §6 attribute the lift to a single change at a time.

---

5. Experimental Setup

5.1 Hardware and software

CUDA-only training. PyTorch 2.x with `torch.cuda` enforced (`scripts/{06,07,12,13}*.py` raise if CUDA is unavailable). HuggingFace `transformers` for CLIP vision encoding. Runs are GPU-pinned via `--gpu_id`.

5.2 Training protocols

Every training script supports the same pair of CLI knobs:

```
--protocol  {default | calibrated4way}
--run_id    <unique tag>            # auto-generated if omitted
--resume_from <checkpoint.pt>       # warm start
```

Outputs are written to `data/processed/experiments/<model_id>/<protocol>/<run_id>/`, with `runs_index.json`, `latest.json`, `best.json` pointers maintained by `scripts/model_registry.py`.

5.3 Evaluation protocols

We always report **masked macro-F1** (per-class F1 on rows where `mask=1`, then averaged across the 7 classes — `common_multilabel.f1_from_counts` and `scripts/06,07,12,13 masked_macro_f1`).

Two thresholding modes are reported side-by-side:

- **`@0.5`**: $\hat{y}_i = \mathbf{1}[\hat{p}_i \geq 0.5]$ for every class.
- **`@per_class_thr`**: per-class thresholds picked by `scripts/08_tune_thresholds.py` via a grid sweep `t∈{0.05, 0.10, …, 0.95}` that maximises class F1 on a *calibration* prediction set.

The leakage-free `calibrated4way` protocol is the recommended one and is the protocol used to declare the *best* model in this report. It enforces:

1. tune `per_class_thresholds.json` **only** on `calib_predictions.json`,
2. apply those frozen thresholds to `val_predictions.json` and `test_predictions.json` via `scripts/09_evaluate_test.py`,
3. compare models on the resulting `test_metrics_calibrated.json`.

Conceptually this is **hold-out threshold calibration** analogous to reserving part of labeled data purely for deploying a decision rule—a standard guard against inflated metrics when optimisation and reporting coincide (Varma & Simon, 2006; Cawley & Talbot, 2010; full entries in Appendix C).

5.4 Hyperparameters and selection

Best-checkpoint selection is `--best_metric val_bce` everywhere by default. We deliberately do **not** select on `val_macro_f1@thr` because (a) `@0.5` macro-F1 is degenerate for a miscalibrated VLM and (b) selecting on `@thr` re-introduces a soft form of threshold-tuning leakage. Cosine LR with 2-epoch linear warmup, `min_lr=1e-6`, gradient-norm clip 1.0, AdamW with `weight_decay=1e-4`, seed 42 throughout.

5.5 Reproducibility

End-to-end pipeline:

```
# One-shot (recommended): splits, graphs, dual CLIP caches, all models, calibrated + le
./scripts/reproduce_all_results.sh
# Artifact layout: RUN_ID=<tag> appears under data/processed/experiments/<model_id>/<pr
```

Or step-by-step (must use **separate** CLIP caches for `default` vs `calibrated4way` rows so path order matches the rows JSON):

```
python scripts/01_build_canonical_labels.py
python scripts/02_align_vlm_outputs.py
python scripts/03_make_multilabel_splits.py
python scripts/03_make_multilabel_splits_4way.py
python scripts/04_build_coerror_graph.py --train_rows_json data/processed/splits/train_
python scripts/04_build_coerror_graph.py --train_rows_json data/processed/splits_4way/t
python scripts/05_run_baseline_frozen_vlm.py ...
python scripts/06_run_baseline_mlp.py ...
python scripts/07_train_gnn_adapter.py ...
python scripts/12_train_clip_vlm_gnn_adapter.py --clip_cache_pt data/processed/embeddin
python scripts/12_train_clip_vlm_gnn_adapter.py --clip_cache_pt data/processed/embeddin
python scripts/13_train_bipartite_gnn_adapter.py --clip_cache_pt ...  # mirror 12 — def
python scripts/08_tune_thresholds.py ...
python scripts/09_evaluate_test.py ...
python scripts/11_package_report.py
```

Canonical comparison numbers in §6 mirror **`reports/comparison/overall.json`** produced by **`python scripts/11_package_report.py`**, aggregated from **`best.json`** → `.../<model_id>/<protocol>/repro_full_20260503/` (tables round to four decimals).

Run inventory: `data/processed/experiments/<model_id>/<protocol>/{runs_index,latest,best}`
Reports: `reports/comparison/overall.md`, `reports/comparison/overall.json`, `reports/gnn_adapter/report.md`. Legacy checkpoints and superseded caches may live under `data/processed/experiments/_archive_20260503/` if you archived older runs.

---

6. Results

All numbers below are macro-F1, rounded to 4 decimals; raw 6-decimal values are in `runs_index.json` and the `metrics.json` of each run directory. The `default` columns use the 3-way split; the `calibrated4way` column uses the leakage-free 4-way split with thresholds tuned on `calib`.

6.1 Main comparison

Model	Default Val @0.5	Default Test @0.5	Default Val @thr	Default Test @thr	Calib4way Val	**Calib4way Test**
`vlm_zeroshot` (frozen VLM)	0.0473	0.0472	NA	NA	0.6513	**0.6512***
`vlm_mlp` (MLP adapter)	0.5208	0.5191	NA	NA	0.6548	**0.6544**
`gnn07_label_residual`	0.0442	0.0423	0.0442	0.0423	0.6513	**0.6512**
`gnn12_clip_vlm_mlp`	0.6095	0.6013	0.6095	0.6013	0.6792	**0.6777**
`gnn13_clip_bipartite`	0.6542	0.6371	0.6542	0.6371	0.6923	**0.6889**

* **`Calib4way`** column: macro-F1 after `08_tune_thresholds.py` on `calib_predictions.json` only, then evaluated on **val/test** predictions with those frozen thresholds (`test_metrics_calibrated`). For **`vlm_zeroshot`**, predictions are the raw frozen VLM probabilities (`x_probs`) exported via `scripts/export_rows_to_predictions.py`; this is **not** the same numeric story as $\hat{y} = \mathbb{1}[p \geq 0.5]$, where macro-F1 is ≈ 0.047 (near-trivial negatives). Detailed `@0.5` JSON for the auxiliary baseline path is still in `data/processed/experiments/baseline_frozen_`

Headline result. Under the leakage-free calibrated protocol (`reports/comparison/overall.json`), `gnn13_clip_bipartite` reaches **macro-F1 = 0.6889** on test (**+3.4** macro-F1 **points** vs MLP **0.6544**, **+1.1** vs `gnn12` **0.6777**). Calibrated zeroshot, residual GNN, and **MLP cluster near 0.6512**, leaving bipartite **+0.0377** calibrated macro-F1 over that reference (**0.6512** $\rightarrow$ **0.6889**). On the **`default`** splits, masking at $t = 0.5$ leaves frozen zeroshot at ≈ 0.047 macro-F1, while trained heads move into the ≈ 0.5–0.65 `@0.5` band (same table).

6.2 The threshold-tuning leakage trap (RCA)

Look at `gnn07_label_residual` on the replicated **`default`** run (`repro_full_20260503`): at fixed threshold 0.5, masked macro-F1 on **validation** stays near **0.044** (`metrics.json`).

If thresholds are tuned on **`val_predictions.json`** and evaluated **again on validation** (`…/val_metrics_thr_tuned_on_val_LEAKY.json`), macro-F1 on that same split climbs to ≈ 0.657 — an $\approx +0.61$ artefact from re-using the same labelled split both to pick thresholds and to report the headline number. Mechanism:

1. The residual adapter learns a near-zero-mean correction Δ on top of frozen VLM logits whose distribution is roughly $\mathcal{N}(\mu \ll 0, \sigma)$ for non-`No Finding` classes (Qwen2-VL's soft *negative* bias).
2. Sigmoid of those logits hugs ≈0.05–0.20 for true positives. At $t = 0.5$ virtually no class fires → recall ≈0 → F1 ≈0.
3. Threshold sweep $t \in \{0.05, \ldots, 0.95\}$ recovers each class's F1 by simply picking a low threshold; tuning *and* reporting on the same split optimistically samples the F1-maximising operating point.

Compare this to the leakage-free 4-way protocol on the same model (`test_metrics_calibrated.jso` **0.6512** on test in the replicated registry—that is the leakage-free calibrated score, and **it matches calibrated frozen probabilities** (**0.6512**) in this artifact bundle. The qualitative lesson persists: Δ logits are saturated at $t = 0.5$, whereas per-threshold optimisation on p restores recall; tuning those thresholds **on evaluation data** exaggerates perceived lift. Always tune thresholds on a disjoint `calib` split reserved for scoring rules alone.

6.3 Effect of CLIP image features (M2 → M3)

Holding the homogeneous label graph fixed and adding a frozen CLIP branch lifts calibrated test F1 from **0.6512 → 0.6777** (**+2.65** macro-F1; same calibrated reference as **`gnn07`** / zeroshot in this sweep). CLIP embeddings appear **complementary** to frozen VLM scores on the bipartite/co-error heads even when logits are calibrated post hoc.

6.4 Effect of bipartite topology (M3 → M4)

Replacing the $C \times C$ homogeneous graph with **bipartite attribute → object** flow lifts calibrated test macro-F1 from **0.6777 → 0.6889** (**+1.12** vs `gnn12` in **`reports/comparison/overall.json`**). The qualitative structural argument—a single imaging "object" and many latent findings—remains the same (`scripts/gnn_bipartite.py`), with **`mode=all`** default edge weights unless `vlm_positive` is tuned.

6.5 Per-protocol observations

- The MLP baseline closes most of the gap to the residual GNN under the calibrated protocol; the structural prior (label graph) does **not** add measurable value on top of class-bias correction *unless* image features are also re-injected.
- **`gnn12` / `gnn13`** gain substantially when thresholds are constrained to **`calib`**: compare default split test macro-F1 at reported per-class thresholds (same split as tuning in that column—**still optimistic**) **0.6013**, versus leakage-free calibrated test **0.6777 / 0.6889**.

- Zeroshot **@0.5** remains ≈ 0.047 macro-F1 (test)—the pathology of uniformly high thresholds against negatively biased logits. With **honest calibrated** thresholds on frozen probabilities only, zeroshot reaches ≈ 0.6512, and the best bipartite head still delivers $\approx +0.038$ macro-F1 on top; **trained** adapters achieve usable masked macro-F1 at **`@0.5`** on the defaults split (§6.1 first columns).

---

7. Ablations and Analysis

7.1 Edge mode in the bipartite GNN (`--edge_mode {all, vlm_positive}` × `--vlm_tau`)

`vlm_positive` masks out attributes whose VLM probability is below `vlm_tau`, with an all-ones fallback for empty rows. In our runs `mode=all` was used as the default; `mode=vlm_positive` adds a per-row sparsification that is most useful when the VLM is well-calibrated for *some* classes (`No Finding`, `Cardiomegaly`) and noisy for others (`Pneumonia`). It is exposed in `scripts/13_train_bipartite_gnn_adapter.py` for dataset-specific tuning.

7.2 Number of bipartite layers

`--gnn_hidden_dims 512,256` (i.e., $L = 2$) is the default. The first layer encodes high-frequency attribute → object updates, the second consolidates them. Going deeper (e.g., `512,256,128`) does not help because the bipartite graph has diameter 2 and additional layers only re-mix the same object state.

7.3 Co-error graph `top_k` and `τ`

`top_k=3, τ=0.02` was used as the default. Lower `τ` introduces noisy edges (rare co-errors); higher `top_k` densifies the graph and effectively averages neighbours, which hurts the residual GNN's precision on rare classes (`Pneumonia`, `Consolidation`). The `gnn07` configuration is *small*, so over-densifying the graph quickly approaches a global mean and erases per-class structure.

7.4 `alpha` (residual scale)

All three GNNs default to `alpha=0.5`. Setting `alpha=0` reduces the model to a pure label-graph predictor with no VLM anchor and underperforms (`gnn07` cannot recover from the VLM's negative bias without the residual). Setting `alpha=1.0` makes the adapter a strict additive correction and is more stable but slightly less accurate than `0.5` on our splits.

7.5 Best-checkpoint metric (`--best_metric`)

`val_bce` is the default. `val_macro_f1_05` selects on the same metric we report at `@0.5` and biases the model toward overconfident logits. `val_macro_f1_thr` selects on per-class-tuned val F1 and is the strongest in-sample metric, which is exactly why we **do not** use it: it is one threshold-tuning step away from the same leakage as §6.2.

7.6 Class-weighted loss (`pos_weight_max=100`)

The 7 CheXpert classes are extremely imbalanced (`Pneumonia` ≈ 2% positive in our train split, `No Finding` ≈ 50%). Capping `pos_weight` at 100 prevents a single rare class from dominating the gradient. Removing the cap (`pos_weight_max=∞`) destabilises early training and makes the cosine schedule diverge.

7.7 The flat-MLP control

The MLP baseline already receives the *full* `[logits; probs]` vector and spans per-class affine-ish recalibration. It scores **0.6544** on calibrated test (**`reports/comparison/overall.jso`**) so calibrated macro-F1 gains **above that line** (> 0.6544) are evidence of *cross-class structure* (CLIP + graph) rather than scalar recalibration alone.

---

8. Discussion

8.0 Framing: when to prefer a GNN adapter over VLM fine-tuning

Use **adapter-only domain adaptation** (no VLM gradients) when: (i) you cannot afford full fine-tuning compute or liability of changing a shared foundation model; (ii) you need **fast iteration** on label policies, thresholds, or hospital-specific biases; (iii) you already have **offline VLM scores** and want a reproducible head. Prefer a **GNN head** over a flat MLP when you believe **label structure** (co-occurrence, co-error, or image–attribute factorization) should be **baked in** rather than re-learned from limited data—and when you can supply a graph prior (co-error from train) or a bipartite template (attribute→object). Prefer **full VLM fine-tuning** when the domain gap is primarily **visual** (e.g., completely new modality or resolution) and no auxiliary frozen image encoder closes that gap; this repo's best results suggest that **re-injecting CLIP** captures part of that visual gap *without* touching the VLM.

8.1 What actually drives the gains

In ascending order of contribution to **0.6889** calibrated bipartite macro-F1 **vs** calibrated frozen probs (≈ 0.6512) and naïve **@0.5**:

1. **Class-weighted masked BCE plus honest threshold protocol** (vs naïve `@0.5`): lifts calibrated frozen probabilities from $0.047 \rightarrow\approx 0.65$ macro-F1 (zeroshot; see §6.1 footnote on `export_rows_to_predictions + calib-only` thresholds) and aligns adapter scores with decision-rule hygiene.
2. **CLIP image features re-injected** (M2 → M3 vs the same ≈ 0.6512 calibrated reference): **+2.65** macro-F1 (to **0.6777**) under `08/09`.
3. **Bipartite attribute → object topology** (M3 → M4): +1.1 F1 by matching the data-generating process.
4. **Co-error label graph** (M1 → M2): essentially neutral once calibration is honest. The graph encodes a prior that the adapter can also learn from data given enough capacity.

8.2 What does *not* drive gains

- The choice of `gnn07` adjacency normalization does not matter beyond row-stochastic.
- Going past 2 bipartite layers does not improve results.
- Selecting checkpoints on val-thr macro F1 inflates val numbers without a matching test improvement.

8.3 Practical recommendation

For a production system that must keep the foundation VLM frozen, **`gnn13_clip_bipartite` evaluated under the 4-way calibrated protocol** is the recommended configuration (**0.6889** calibrated test macro-F1 in `reports/comparison/overall.json`). If CLIP inference is unacceptable, **`vlm_mlp` under that same protocol remains competitive** (**0.6544**) at lower architecture complexity.

---

9. Limitations

1. **Single VLM source.** All non-zeroshot adapters consume Qwen2-VL-2B-Instruct outputs; we have not measured robustness under a different VLM.
2. **Single CLIP backbone.** `openai/clip-vit-base-patch32` is the smallest CLIP and is not pretrained on chest X-rays; a domain-adapted encoder (e.g., BiomedCLIP) would likely lift M3/M4 further.
3. **Single random seed.** All numbers are seed-42; we have not reported variance across seeds.
4. **Patient-grouped but not site-grouped splits.** CheXpert is single-site, so distribution shift is not measured here.
5. **Macro-F1 only.** AUROC, balanced accuracy and per-class recall at fixed precision would round out the comparison.
6. **Threshold grid is coarse** (`step=0.05`); a finer grid or differentiable-threshold method (e.g., Fβ-soft) might shave a small additional F1.

---

10. Reproducibility Summary

- Code: `scripts/01–13`, `scripts/common_multilabel.py`, `scripts/model_registry.py`, `scripts/gnn_bipartite.py`.
- Configs: `configs/{data,graph,train_gnn,train_clip_gnn,eval}.yaml`.
- Seeds: 42 throughout (`set_seed`).
- Splits: `data/processed/splits/` (3-way) and `data/processed/splits_4way/` (4-way).
- Graph: `data/processed/graph/{edge_index,edge_weight}.json`.
- CLIP caches: `data/processed/embeddings/clip_vitb32_default.pt` and `clip_vitb32_calibrated4way.pt` (one per split regime; **never** mix row sets in a single cache file).
- Run registry: `data/processed/experiments/<model_id>/<protocol>/{runs_index,latest,be`

- Reports: `reports/gnn_adapter/report.md`, `reports/comparison/overall.{md,json}`.
- Inference UI: `gradio_inference.py` (resolves `best.json` under `.../<model_id>/<default|cali` optional legacy weights may live in `.../_archive_20260503/` after cleanup).

---

11. Conclusion

We argue that **GNN-based adapters remain a practical recipe for domain adaptation in multi-label CXR classification without fine-tuning the VLM**: the foundation model supplies fixed (z, p) scores (CLIP optional), while a lightweight head learns **structured, label-aware corrections**. On the replicated registry bundle (**`RUN_ID=repro_full_20260503`**, `reports/comparison/overall.json`), bipartite **`gnn13_clip_bipartite`** attains **0.6889** calibrated test macro-F1—**≈ +3.4** over calibrated **`vlm_mlp` (0.6544)** and **≈ +3.8** over calibrated frozen probabilities alone (0.6512), while t=0.5 keeps masked macro-F1 near **0.05** on zeroshot without threshold optimisation. Threshold tuning **on evaluation data** still inflates **validation** residuals by $\approx +0.61$ macro-F1 (see `*_LEAKY.json` / §6.2); reserving **`calib`** removes that ambiguity. Jointly these facts support **"freeze VLM, adapt with graphs (and calibrated decisions)"** whenever full encoder fine-tuning is prohibitive.

Appendix A — Script-to-Artifact Map

Stage	Script	Primary artifacts
Canonical labels	`01_build_canonical_labels.py`	`data/processed/multilabel/canonical_`
VLM alignment	`02_align_vlm_outputs.py`	`data/processed/multilabel/aligned_v`
3-way splits	`03_make_multilabel_splits.py`	`data/processed/splits/{train,val,tes`
4-way splits	`03_make_multilabel_splits_4way.py`	`data/processed/splits_4way/{train_fi`
Co-error graph	`04_build_coerror_graph.py`	`data/processed/graph/` (defaults) plus `data/processed/graph_4way/` (4-way splits)
Frozen VLM eval	`05_run_baseline_frozen_vlm.py`	`.../baseline_frozen_vlm/metrics.jso`
MLP baseline	`06_run_baseline_mlp.py`	`.../vlm_mlp/<protocol>/<run_id>/{me`
Residual GNN	`07_train_gnn_adapter.py`	`.../gnn07_label_residual/<protocol>/`
Threshold tuning	`08_tune_thresholds.py`	`.../<protocol>/<run_id>/per_class_t` (calib-fed) or `*_tuned_on_val_LEAKY.json`
Evaluation	`09_evaluate_test.py`	`.../<run_id>/test_metrics_calibrate` (honest) or `*_LEAKY.json` (diagnostic)
Ablation collation	`10_run_ablations.py`	`data/processed/experiments/ablations` (writes new dir if missing)
Markdown report	`11_package_report.py`	`reports/{gnn_adapter/report.md, compari-son/overall.{md,json}}`

Stage	Script	Primary artifacts
CLIP+VLM GNN	`12_train_clip_vlm_gnn_adapter.py`	`runs/gnn12_clip_vlm_homo/<protocol>/<`
Bipartite GNN	`13_train_bipartite_gnn_adapter.py`	`runs/gnn13_clip_bipartite/<protocol>/`

Appendix B — Default Hyperparameters

Model	Hidden	Layers	LR	Sched	Epochs	Batch	Dropout	α	Other
`vlm_mlp`	64	2	1e-3	none	20	full	0.1	—	AdamW, wd 1e-4, pos_weight≤100
`gnn07_label_residual`	32	1 mes-sage-pass	3e-4	cosine + warmup 2	≤80 (es 18)	full	0	0.5	grad_clip 1.0, val_bce check-point
`gnn12_clip_vlm_homo`	64	K=2 GNN	3e-4	cosine + warmup 2	≤60 (es 16)	32	0	0.5	CLIP B/32 frozen, cached em-bed-dings
`gnn13_clip_bipartite`	[512, 256]	1 or 2 bipar-tite	3e-4	cosine + warmup 2	≤60 (es 16)	32	0.2	0.5	object_dim 512, edge_mode=all

Appendix C — References

Grouped by topic for quick lookup; alphabetical within each topic.

C.1 Dataset & task

- Irvin J., Rajpurkar P., Koh M., Yu Y., Cicurel S., Chute C., … Ng A. Y. (2019). *CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison*. Proceedings of AAAI Conference on Artificial Intelligence (AAAI).

C.2 Vision–language foundation models & parameter-efficient tuning

- Radford A., Kim J. W., Hallacy C., Ramesh A., Goh G., Agarwal S., … Sutskever I. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. Proceedings of ICML.

- Wang P., Bai S., Tan S., Wang S., Fan Z., Bai J., … Zhou J. (*Qwen team*) (2024). *Qwen2-VL: Enhancing Vision-Language Model's Perception of the World at Any Resolution*. arXiv:2409.12191 https://arxiv.org/abs/2409.12191.
- Hu E. J., Shen Y., Wallis P., Allen-Zhu Z., Li Y., Wang S., Wang L., Chen W. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. Proceedings of ICLR.
- Houlsby N., Giurgiu A., Jastrzebski S., Morrison Q., Larochelle H., Gesmundo M., Attariyan H., Gelly S. (2019). *Parameter-Efficient Transfer Learning with NLP Adapter Modules*. Proceedings of ICML.

C.3 Graph neural networks — homogeneous convolution & message passing

- Kipf T. N., Welling M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR poster.
- Hamilton W., Ying Z., Leskovec J. (2017). *Inductive Representation Learning on Large Graphs*. NeurIPS.
- Veličković P., Cucurull G., Casanova A., Romero A., Liò P., Bengio Y. (2018). *Graph Attention Networks*. ICLR.
- Gilmer J., Schoenholz S. S., Riley P., Vinyals O., Dahl G. E. (2017). *Neural Message Passing for Quantum Chemistry*. ICML. (Defines a general pairwise message/update framework that subsumes many bipartite and heterogeneous aggregations.)

C.4 Heterogeneous, relational & bipartite-style graphs

- Schlichtkrull M., Kipf T. N., Bloem P., van den Berg R., Titov I., Welling M. (2018). *Modeling Relational Data with Graph Convolutional Networks*. European Semantic Web Conference (ESWC). https://doi.org/10.1007/978-3-319-93417-4_38
- Wang X., Ji H., Shi C., Wang B., Ye Y., Cui P., Yu P. S. (2019). *Heterogeneous Graph Attention Network*. The Web Conference (WWW). https://doi.org/10.1145/3308558.3313562

C.5 Multi-label image recognition via label graphs

- Chen Z.-M., Wei X.-S., Wang P., Guo Y. (2019). *Multi-Label Image Recognition with Graph Convolutional Networks*. Proceedings of IEEE/CVF CVPR.

C.6 Threshold choice, probabilistic calibration & "leakproof" evaluation protocol

Choosing per-class thresholds and reporting accuracy/F1 mixes **classification calibration** with **selection bias** whenever the same labeled split is reused for tuning and leaderboard reporting.

- Lipton Z. C., Elkan C., Narayanaswamy B. (2014). *Optimal Thresholding of Classifiers to Maximize F1 Measure*. In *Proceedings of ECML PKDD 2014*

(Springer LNCS vol. 8726), pp. 225–239. https://doi.org/10.1007/978-3-662-44851-9_15 (related preprint: *Thresholding Classifiers to Maximize F1 Score*, arXiv:1402.1892.)

- Lewis D. D. (1995). *Evaluating and optimizing autonomous text classification systems*. ACM SIGIR. (Classic framing of precision/recall trade-offs via thresholds.)
- Platt J. C. (1999). *Probabilistic outputs for Support Vector Machines and comparisons to regularized likelihood methods*. In *Advances in Large Margin Classifiers* (MIT Press). (Platt / temperature-style scaling lineage.)
- Niculescu-Mizil A., Caruana R. (2005). *Predicting Good Probabilities with Supervised Learning*. ICML.
- Guo C., Pleiss G., Sun Y., Weinberger K. Q. (2017). *On Calibration of Modern Neural Networks*. Proceedings of ICML.
- Varma S., Simon R. (2006). *Bias in Error Estimation When Using Cross-Validation for Model Selection*. BMC Bioinformatics, 7(1), 91. https://doi.org/10.1186/1471-2105-7-91
- Cawley G. C., Talbot N. L. C. (2010). *On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation*. Journal of Machine Learning Research (JMLR), 11, 2079–2107. http://jmlr.org/papers/v11/cawley10a.html
- Dietterich T. G. (1998). *Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms*. Neural Computation, 10(7), 1895–1923. (Multiple runs / paired tests when comparing adapters.)

Together, Varma & Simon (2006) and Cawley & Talbot (2010) underpin the methodological point that **any post hoc rule fit on labeled data—including per-class thresholds—must consume a disjoint hold-out** if the same numerical split is otherwise used for model comparison; our **`calib`** split instantiates exactly that separation from `train_fit`, `val`, and `test`.